

Firestore in the Fashion Store App

How Core, Auth, Firestore, Storage, and App Check connect in code — with file locations and reasons.

1. Pieces of the stack

Firestore Core — Initializes Firestore once; supplies project id, API keys, and storage bucket per platform (via `firebase_options.dart`).

Firestore Auth — Email/password sign-in, registration, password reset, and listening for who is signed in.

Cloud Firestore — Products, promo banners, user profiles, wishlists, and orders as documents and subcollections.

Firestore Storage — Binary files (e.g. profile images); catalog can store a Storage path and resolve it to a download URL.

App Check (release builds) — Helps ensure only your real app uses your backend; debug builds skip it for speed.

2. Dependencies (`pubspec.yaml`)

Packages: `firebase_core`, `firebase_auth`, `cloud_firestore`, `firebase_storage`, `firebase_app_check`. Reason: each Firestore product is a separate Flutter plugin.

3. Bootstrap — where Firestore starts

`lib/main.dart` — `WidgetsFlutterBinding.ensureInitialized()`; then `Firestore.initializeApp(options: DefaultFirestoreOptions.currentPlatform)`. On failure, shows an error screen instead of crashing. In release, activates App Check.

`lib/firebase_options.dart` — Generated (e.g. FlutterFire CLI): `FirestoreOptions` for Android, iOS, Web, etc. Reason: one codebase, multiple platforms.

Teaching point: initialize once in main; elsewhere use `FirestoreAuth.instance`, `FirestoreFirestore.instance`, `FirestoreStorage.instance`.

4. Authentication

lib/widgets/auth_gate.dart — Listens to `authStateChanges()`; signed in → Home, signed out → Login. Uses `AuthFlowFlags` so registration does not flash Home before sign-out.

lib/screens/auth/login_screen.dart — `signInWithEmailAndPassword`; password reset email.

lib/screens/auth/register_screen.dart — `createUserWithEmailAndPassword`; profile setup; then `signOut` so the user signs in manually.

Reason: Auth provides uid used in Firestore paths like `users/{uid}/...`

5. Firestore — collections and code

Area	Where in code	Purpose
products	<code>catalog_repository.dart</code> , <code>admin_panel_screen.dart</code>	Catalog; query <code>active == true</code>
banners	<code>catalog_repository.dart</code> , <code>admin_panel_screen.dart</code>	Promo slides; active filter
<code>users/{uid}</code>	<code>firebase_data_service.dart</code> , <code>profile_screen.dart</code>	Profile: email, <code>displayName</code> , phone, <code>updatedAt</code>
<code>users/{uid}/favorites</code>	<code>wishlist_provider.dart</code>	Wishlist doc id = <code>productId</code>
<code>users/{uid}/orders</code>	<code>checkout_screen.dart</code> (write), <code>orders_screen.dart</code> (read)	Orders after checkout; stream + <code>orderBy createdAt</code>

Reason: global collections for the store; per-user subcollections under `users/{uid}` match typical security rules (only that user writes their data).

UI often uses snapshot streams for live updates when data changes in the console or admin.

6. Firebase Storage

lib/services/firebase_data_service.dart — `uploadProfileImage`: `ref('users/{uid}/profile.jpg')`, `putData`, `getDownloadURL`.

lib/services/catalog_repository.dart — `_resolveImage`: if `imageUrl` is empty, uses `imagePath` with `Storage ref().getDownloadURL()`; caches URLs in memory.

Reason: large binaries belong in Storage; Firestore holds metadata and URLs or paths.

7. End-to-end flow (story)

- App starts → `Firebase.initializeApp` + options.
- `AuthGate` → Login or Home from Auth state.
- `CatalogRepository` → Firestore products & banners; Storage for image paths if needed.
- Register → Auth user + `upsertUserProfile` on `users/{uid}`; sign out → login manually.
- Wishlist → `users/{uid}/favorites`.
- Checkout → add order under `users/{uid}/orders`; Orders screen listens to that path.

8. Security (conceptual)

The Flutter app calls Firebase APIs; **Firestore Security Rules** and **Storage Rules** in the Firebase Console enforce who can read/write. The data layout (`users/{uid}/...`) is designed to align with `request.auth.uid`.

9. Outside Dart (mention in lecture)

Android: `google-services.json`. iOS: `GoogleService-Info.plist`. These work together with `firebase_options.dart` from `FlutterFire`.